

utils

Helper utilities.

DEFAULT_SPARK_APP_NAME module-attribute

```
DEFAULT_SPARK_APP_NAME = 'pySparkJob-created-by-ds_api'
```

logger module-attribute

```
logger = getLogger(__name__)
```

ResourceSearchTypeEnum

Bases: `str`, `Enum`

Procedures to search for given resource within a parent resource.

DESC class-attribute instance-attribute

```
DESC = 'desc'
```

ID class-attribute instance-attribute

```
ID = 'id'
```

OBJECT class-attribute instance-attribute

```
OBJECT = 'object'
```

camel_to_snake

```
camel_to_snake(string: str) -> str
```

Convert camel case to snake case.

Excludes strings starting with underscore, or in upper case eg: ["createObj", "create_obj", "**dict**", "CONSTANT"] = ["create_obj", "create_obj", '**dict**', 'CONSTANT']

Parameters:

Name	Type	Description	Default
<code>string</code> 	<code>str</code>	input string	<i>required</i>

Returns:

Type	Description
<code>str</code>	snake case string

create_spark_app_name [↗](#)

```
create_spark_app_name() -> str
```

Creates an application name for spark.

The created name will take in consideration the following aspects:

1. If the env variable `ds_api.USER_NAME_ENV` is defined, then it will be used as part of the created name. This environment variable is assumed to contain the name of the user that spawned the current process.
2. If the env variable in 1) is not defined, then the env variable `ds_api.USER_ID_ENV` will be used (if defined) as part of the created name. This environment variable is assumed to contain the identifier of the user that spawned the current process.
3. If the env variable in 2) is not defined, then this function calls `os.getpid`, and uses such pid in the created application name.

Returns:

Type	Description
<code>str</code>	A spark application name.

get_optional_or_fail [↗](#)

```
get_optional_or_fail(optional, fail_message, mapper=None)
```

get_optional_or_none [↗](#)

```
get_optional_or_none(java_optional, mapper=None)
```

optional_char_or_else [↗](#)

```
optional_char_or_else(value, or_else)
```

to_java_list [↗](#)

```
to_java_list(pylist)
```

to_java_map [↗](#)

```
to_java_map(pymap)
```

to_java_set [↗](#)

```
to_java_set(pylist)
```

to_python_list [↗](#)

```
to_python_list(java_list, mapper=None)
```

to_python_map [↗](#)

```
to_python_map(java_map, key_mapper=None, value_mapper=None)
```

to_python_set [¶](#)

```
to_python_set(java_set, mapper=None)
```

transform_java_object [¶](#)

```
transform_java_object(java_object)
```